

KUBEEDGEINFER

A Closed-Loop, eBPF-Driven Kubernetes Framework for Heterogeneous Distributed LLM Inference

Turning the one-time model split into a continuous, self-correcting decision driven by kernel-level measurements.

eBPF / CO-RE

Kubernetes operator

Pipeline parallelism

Consumer edge hardware

Research Presentation

Author: • Institution: • Date:

Edge LLM inference is a moving target

1 Why split at all

A 7B+ model does not fit on one consumer device. Pipeline parallelism spreads contiguous layer ranges across several machines, so the split point is the central performance decision.

2 Why the edge is different

Datacenter schedulers assume stable, homogeneous, well-cooled nodes. An edge cluster is laptops and hobby GPU boxes on WiFi: they thermally throttle, their links wander, and they disappear.

3 What existing frameworks do

They profile once, solve the placement problem once, and never revisit it. The plan is optimal for the machine state at $t = 0$ and progressively wrong afterwards.

The consequence

2.4x

bottleneck-stage cost inflicted by a single throttled node under a frozen split

4.3x

when a degraded link and a throttled GPU coincide

0

requests served once a node dies and nothing re-plans

Partitioning and placement

Contribution

Limitation relevant to this work

EdgeShard

IEEE IoT-J, 2024

arXiv:2405.14371

Joint device selection + layer partition by dynamic programming across collaborative edge devices. Up to 50% latency reduction, 2x throughput on Llama2 over 15 physical devices.

- Profiles device speed and bandwidth offline; the DP runs on that static profile.

Galaxy

IEEE INFOCOM, 2024

arXiv:2405.17245

Hybrid tensor/sequence parallelism for in-situ Transformer inference, with heterogeneity-aware planning and tile-based compute/communication overlap. Up to 2.5x lower latency.

- Planning is a one-shot offline step; no runtime re-planning under drift.

Helix

ACM ASPLOS, 2025

arXiv:2406.01566

Models heterogeneous GPU serving as max-flow on a weighted graph and solves placement + request scheduling jointly by MILP. Up to 3.3x throughput on 24-42 node clusters.

- Datacenter-scale GPUs; MILP is too heavy for a seconds-scale control loop.

TPI-LLM

arXiv, 2024

arXiv:2410.00531

Argues tensor parallelism beats pipeline on low-resource devices; sliding-window memory scheduler and star-based allreduce keep 70B-scale weights moving through small RAM.

- Optimises memory and collectives, not adaptation to node degradation.

prima.cpp

arXiv, 2025

arXiv:2504.08791

30-70B inference on real home clusters: Halda scheduler co-optimises CPU/GPU workload and device selection; pipelined-ring parallelism hides disk I/O. 5-17x lower TPOT than llama.cpp.

- Scheduling is decided at load time from static device capability.

Adaptation, decentralisation, and eBPF as control

Contribution

Limitation relevant to this work

LLM Partitioning at the Edge

arXiv, 2025

arXiv:2505.02533

Partitions the decoder at attention-head granularity, co-locating each head with its K/V cache and migrating heads when memory tightens. Within 15-20% of an exact solver.

- *Migration is triggered by memory pressure only - not thermal or network state.*

Parallax

arXiv, 2025

arXiv:2509.26182

Inference service over a decentralised pool of volunteer, non-uniform machines, addressing placement and routing without a central trusted cluster.

- *Decentralised placement, but no kernel-level measurement substrate.*

Adaptive Layer Splitting (MBRL)

FITEE, Springer, 2025

doi:10.1631/FITEE.2400468

Model-based reinforcement learning chooses the split point for wireless LLM inference as channel quality varies - explicitly an adaptive, not one-shot, formulation.

- *Learned policy needs training and gives no optimality guarantee; single split point.*

Distributed LLMs & MLLMs Survey

arXiv, 2025

arXiv:2503.16585

Survey of distributed inference across advances, challenges and directions; names dynamic adaptation under heterogeneity as an open problem.

- *Confirms the gap this work targets rather than filling it.*

Agentic OS / sched_ext

arXiv, 2025

arXiv:2509.01245

Custom Linux schedulers loaded at runtime as eBPF programs (sched_ext, Linux 6.12) - evidence that eBPF is becoming a control substrate, not only an observability one.

- *Schedules CPU tasks on one host; no notion of a distributed model pipeline.*

Where the literature stops

Capability	Datacenter serving (Helix)	Edge partitioning (EdgeShard, Galaxy, prima.cpp)	Adaptive splitting (MBRL, head-level)	KubeEdgeInfer
Heterogeneity-aware placement	yes	yes	yes	yes
Exact optimum for a snapshot	MILP	DP	no (learned / heuristic)	DP, unit-tested vs brute force
Re-plans while serving	request routing only	no	yes	yes, every 2 s
Kernel-measured network input	no	user-space probes	channel estimate	eBPF tp_btf/tcp_probe sRTT
Thermal / throttle input	no	no	no	NVML or /sys/class/thermal
Survives node loss	replication	no	no	heartbeat-forced repartition
Applies without restart	n/a	no	no	gRPC hot reassign, generation-fenced

No prior system closes the loop from kernel-level measurement back to the layer split while inference is running.

PROBLEM STATEMENT

Given N transformer layers and K heterogeneous edge nodes whose compute speed, link latency and liveness all vary during inference, continuously maintain a contiguous layer assignment that minimises the bottleneck stage - without restarting the pipeline.

1 Measure

Obtain per-flow latency and per-node thermal state cheaply and without instrumenting the inference code.

2 Decide

Solve the placement exactly at telemetry rate, and suppress oscillation when the measurement is merely noisy.

3 Act

Move layer ranges between live workers without dropping in-flight requests or restarting pods.

4 Heal

Detect node loss and redistribute its layers across the survivors automatically.

Constraint

Commodity Linux only.

No datacenter interconnect, no vendor telemetry agent, no change to the model code, and no assumption that a node stays alive.

What is new here

1 Kernel telemetry as a scheduling input

CO-RE eBPF programs (tp_btf/tcp_probe, fentry/tcp_sendmsg) feed per-flow smoothed RTT straight into a model-placement decision. Prior edge-LLM systems use user-space probes or offline profiles; none drives the partitioner from the kernel's own TCP state.

2 A closed loop, not a one-shot plan

WATCH-DECIDE-ACT-HEAL runs every 2 s for the lifetime of the deployment. The split is a continuously maintained control variable rather than a deployment-time constant.

3 Hysteresis-gated exact optimisation

The $O(N^2 K)$ DP is exact for each snapshot; a 15% improvement threshold plus a 30 s cooldown converts it into a stable controller. One gate handles thermal, network and failure triggers alike.

4 Restart-free, generation-fenced reassignment

Layer ranges move over gRPC while requests are in flight. Every assignment carries a generation; workers reject stale forwards and the stateless design makes context replay trivially correct.

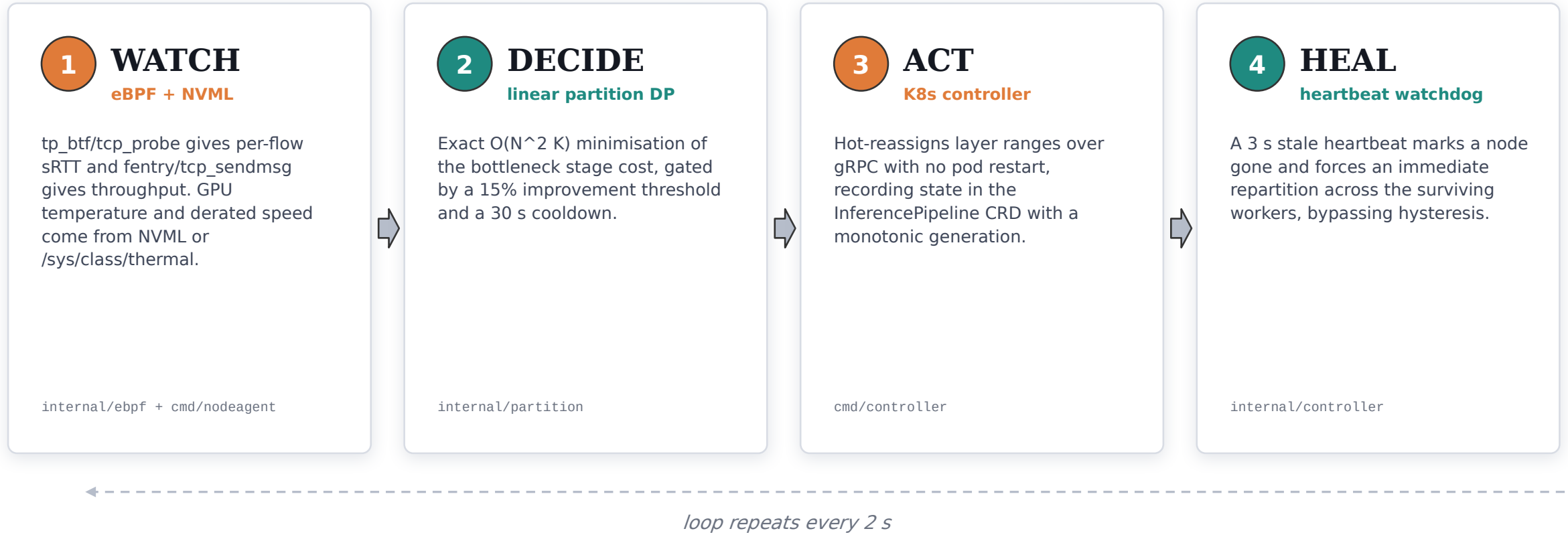
5 An ablation that isolates continuous telemetry

A profile-once mode reuses the same DP and apply path but freezes its inputs, approximating offline-profiling systems - so the measured gain is attributable to live telemetry, not to a better solver.

Core novelty claim

Not a better partitioning algorithm - the same classical DP the literature already uses, placed inside a kernel-measured feedback loop that never stops running.

The closed loop



Consistency: every assignment carries a generation. Workers reject stale-generation forwards; the router refetches the layout and replays the accumulated context. Workers are stateless, so replay is trivially correct.

Formulation and stability

Objective

minimise **max** **cost(i)**
 i in 1..K

$\text{cost}(i) = \text{layers}(i) \times \text{perLayerMs} / \text{speed}(i) + \text{linkMs}(i-1)$

subject to contiguous, non-overlapping ranges covering all N layers. An empty stage costs nothing - so the DP may bypass a worker entirely when its link has degraded past the value of its compute.

Solution

$f[j][w] = \min \text{ over split of } \max(f[\text{split}][w-1], \text{cost}(w, j - \text{split}))$

The classical linear-partition recurrence: minimal bottleneck for the first j layers over the first w workers. $O(N^2 K)$ - about 1.7k operations for $N=12$, $K=3$, so re-solving at 0.5 Hz is free. Verified against brute force on 200 randomised instances.

Why a bare optimiser is not a controller

Telemetry is noisy. Re-applying the argmin on every 2 s tick would thrash the pipeline for sub-millisecond gains. Three rules turn the optimiser into a stable controller:

Improvement threshold

Apply only if the new split beats the current one, re-evaluated under current telemetry, by 15%.

Cooldown

At most one change per 30 s, so a transient spike cannot start an oscillation.

Membership override

A changed worker set bypasses both gates and repartitions immediately - correctness beats stability.

Both thresholds are tunable live via IMPROVEMENT_FRAC and COOLDOWN_S.

Implementation



Node agent

Go + C (CO-RE BPF)

Privileged DaemonSet. Attaches tp_btf/tcp_probe and fentry/tcp_sendmsg, filtered to worker ports, and exports per-flow sRTT plus GPU state through a pluggable gpu.Reader (simulated / cputherm nvml).



Controller

Go, controller-runtime

Aggregates telemetry, runs the Decider, writes assignments into the InferencePipeline CRD and pushes them to workers over gRPC. Watchdog forces repartition on a 3 s stale heartbeat.



Workers + router

Python

Stateless shard servers with pluggable sim and gpt2 backends, on WORKER_DEVICE=auto|cuda|cpu. The router drives stages hub-and-spoke and replays context on a generation mismatch.



Benchmark harness

Python

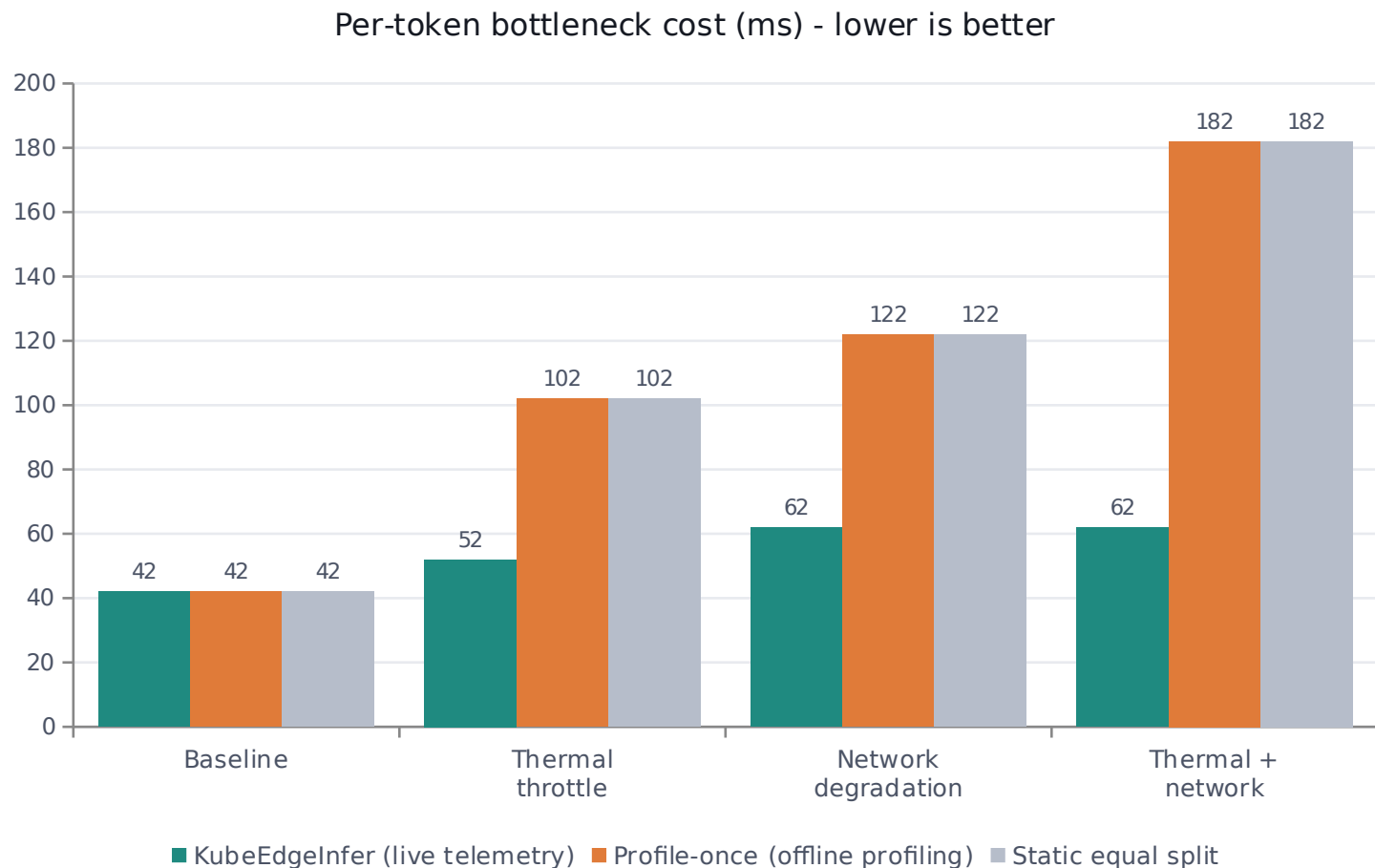
Five fault scenarios x three modes, measuring bubble time, tokens/s and TTFT; plus a hysteresis sweep and a predicted-vs-measured fidelity check.

Deployment reality

- eBPF is real, not simulated - CO-RE against the host BTF, loaded with cilium/ebpf.
- kind gives one host: real eBPF, loopback network, simulated GPU.
- k3s scripts turn laptops and GPU boxes on a LAN into the same cluster with zero YAML editing.
- GPU_MODE=nvml on NVIDIA nodes; GPU_MODE=cputherm reads /sys/class/thermal on GPU-less laptops.
- Distributed GPT-2 output is token-identical to single-process HuggingFace.

RESULTS

Bottleneck-stage cost under injected faults



49%

vs both baselines
under thermal throttle

49%

vs both baselines
under link degradation

66%

vs both baselines
when the two coincide

Node failure: the static split is undefined over a worker set that no longer exists - the pipeline stops. KubeEdgeInfer heals to 62 ms.

RESULTS

What the partitioner actually does

layer ranges assigned to stage 1 / 2 / 3

Baseline

3 healthy stages



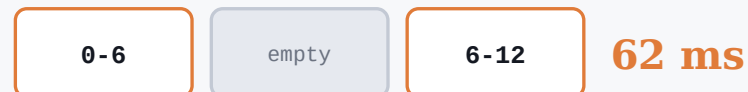
Thermal throttle on stage 2

speed 1.0 -> 0.4



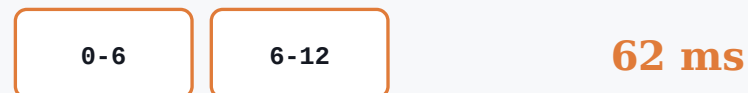
80 ms delay on the hop into stage 2

link 2 -> 82 ms



Stage 3 lost

3 workers -> 2



Stage bypass

Under an 80 ms link the DP assigns stage 2 an empty range. Because an empty stage costs nothing - not even its hop - the router skips it entirely, and the two remaining nodes absorb all 12 layers. The optimiser discovers node exclusion without any special-case rule for it.



Optimality

DP output matches brute force on 200 randomised instances.



Correctness

3-stage distributed GPT-2 is token-identical to single-process greedy decoding.



Model fidelity

Predicted vs measured bottleneck tracks at a stable 3.0-3.24x ratio: a reliable ranking signal, not a calibrated latency.

Against the closest systems

	EdgeShard (IoT-J 24)	Galaxy (INFOCOM 24)	Helix (ASPLOS 25)	prima.cpp (2025)	KubeEdgeInfer
Parallelism	pipeline	tensor + sequence	pipeline + routing	pipelined ring	pipeline
Placement method	DP (exact)	heuristic planner	MILP on max-flow	Halda scheduler	DP (exact)
Telemetry source	offline profile	offline profile	cluster profile	device capability	eBPF + NVML, live
Re-plan while serving	no	no	routing only	no	every 2 s
Handles thermal drift	no	no	no	no	yes
Handles node loss	no	no	replication	no	yes, 3 s watchdog
Target hardware	15 edge devices	edge devices	24-42 GPU nodes	home cluster	consumer edge + k3s
Reported gain	50% latency	2.5x latency	3.3x throughput	5-17x TPOT	49-66% bottleneck*

* Bottleneck-stage cost vs a static split and a profile-once ablation; not directly comparable to the end-to-end figures in the other columns.

CONCLUSION

The split point should be a control variable, not a deployment constant.

KubeEdgeInfer keeps a classical exact partitioner permanently in the loop, fed by kernel-measured latency and real thermal state, and applies its decisions to a running pipeline without a restart. On a heterogeneous edge cluster that is the difference between a plan that was right once and a plan that stays right.

1 Exactness is cheap

The DP costs microseconds; there is no reason to solve it only once.

2 Hysteresis is what makes it usable

One improvement threshold and one cooldown tame thermal, network and failure triggers alike.

3 Kernel telemetry needs no instrumentation

eBPF measures the real transport, not what the application thinks it sent.

Future work

- A Llama-class backend so two 8 GB cards hold a real modern LLM across stages.
- Sustained multi-tenant load to observe genuine NVML throttling, not just flat curves.
- Non-contiguous and attention-head-level assignment, following [arXiv:2505.02533](https://arxiv.org/abs/2505.02533).
- Calibrating the cost model so the DP's bottleneck becomes a latency prediction, not only a ranking.

REFERENCES

All works cited

[1] EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. M. Zhang, J. Cao, X. Shen, Z. Cui. IEEE Internet of Things Journal, 2024. arXiv:2405.14371.

[2] Galaxy: A Resource-Efficient Collaborative Edge AI System for In-situ Transformer Inference. IEEE INFOCOM, 2024. arXiv:2405.17245.

[3] Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. ACM ASPLOS, 2025. arXiv:2406.01566.

[4] TPI-LLM: Serving 70B-scale LLMs Efficiently on Low-resource Edge Devices. 2024. arXiv:2410.00531.

[5] Prima.cpp: Fast 30-70B LLM Inference on Heterogeneous and Low-Resource Home Clusters. 2025. arXiv:2504.08791.

[6] Large Language Model Partitioning for Low-Latency Inference at the Edge. 2025. arXiv:2505.02533.

[7] Parallax: Efficient LLM Inference Service over Decentralized Environment. 2025. arXiv:2509.26182.

[8] Adaptive layer splitting for wireless large language model inference in edge computing: a model-based reinforcement learning approach. Frontiers of Information Technology & Electronic Engineering, Springer, 2025. doi:10.1631/FITEE.2400468.

[9] Distributed LLMs and Multimodal Large Language Models: A Survey on Advances, Challenges, and Future Directions. 2025. arXiv:2503.16585.

[10] Towards Agentic OS: An LLM Agent Framework for Linux Schedulers. 2025. arXiv:2509.01245.

Tooling: cilium/ebpf (CO-RE loader) · Kubernetes controller-runtime · NVIDIA NVML · k3s · Hugging Face Transformers (GPT-2).